C# Ders - 21 Ekim 2026 Ders Notları

1. Metot (Fonksiyon) Nedir ve Neden İhtiyaç Duyarız?

Metotlar; belirli bir görevi yerine getirmek üzere paketlenmiş, ismi olan ve istendiğinde tekrar tekrar çağrılabilen **bağımsız kod bloklarıdır**.

Neden Kullanırız?

- **DRY Prensibi (*Don't Repeat Yourself*):** Aynı kod satırlarını onlarca kez kopyala-yapıştır yapmaktan kurtarır.
- **Modülerlik ve Okunabilirlik:** Yüzlerce satırlık karmaşık bir kodu küçük, anlaşılır parçalara böler.
- **Kolay Bakım:** Bir hesaplama mantığında hata çıktığında 50 farklı yeri değil, yalnızca ilgili metodu düzeltmek yeterlidir.

2. Metotların Anatomisi ve 4 Temel Türü

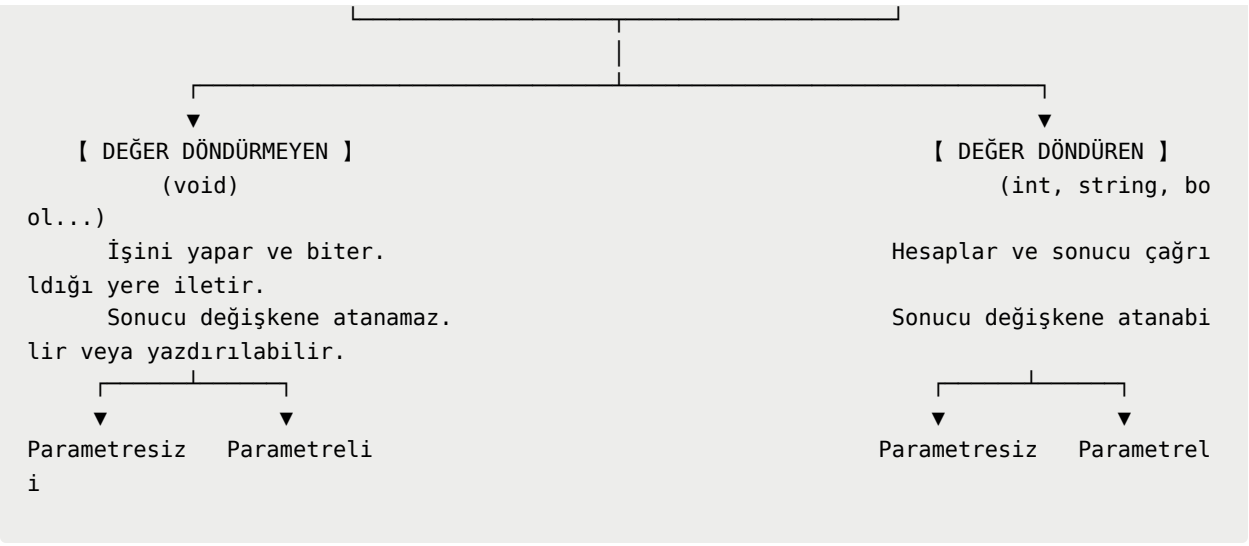
Bir metot tanımlanırken derleyiciye 3 temel bilgi verilir:

1. **Geriye bir sonuç döndürecek mi?** (`void` mi yoksa bir `VeriTipi` mi?)
2. **Metodun adı ne?** (C#'ta kural olarak `PascalCase` yazılır: `SayiTopla`, `MetinOku` vb.)
3. **Çalışmak için dışarıdan veriye ihtiyacı var mı?** (Parantez içi `(parametreler)`)

Plaintext

```
ErişimBelirteci GeriDönüşTipi MetotAdı(ParametreTipi parametreAdi)
{
    // Çalışacak Kodlar
    return deger; // Eğer dönüş tipi void değilse zorunludur!
}
```

```
                    ┌──────────────────────────────────────┐
                    │            METOT TÜRLERİ             │
```



Tür 1: Değer Döndürmeyen ve Parametresiz Metotlar (`void`)

Dışarıdan girdi almaz, geriye bir veri üretmez; sadece içindeki işlemi icra eder.

C#

```
void SayHello()
{
    Console.WriteLine("Hello!!!");
}

// Çağrılışı:
SayHello();
```

Tür 2: Değer Döndürmeyen Ama Parametre Alan Metotlar

Çalışabilmek için dışarıdan parametreye ihtiyaç duyar ancak işi bitince geriye bir değer teslim etmez.

C#

```
void MerhabaDe(string name)
{
    Console.WriteLine("Merhaba " + name);
}

// Çağrılışı:
MerhabaDe("Berker"); // Ekrana: "Merhaba Berker" yazar.
```

Tür 3: Değer Döndüren Ama Parametresiz Metotlar

Dışarıdan bir bilgi almaz ancak çalıştığında çağıran yere bir veri teslim eder. Metot gövdesinde mutlaka `return` anahtar kelimesi yer almalıdır.

C#

```
string MerhabaDemeli()
{
    return "Merhaba"; // string tipinde bir veri fırlatır/döndürür
}

// Çağrılışı:
string gelenMesaj = MerhabaDemeli(); // Dönen değer değişkene aktarılabilir
Console.WriteLine(gelenMesaj);
```

Tür 4: Değer Döndüren ve Parametre Alan Metotlar

En esnek ve en yaygın kullanılan türdür. Girdileri alır, işler ve sonucu çağrıldığı satıra `return` ile teslim eder.

C#

```
int SayiAl(string mesaj)
{
    Console.WriteLine(mesaj);
    int.TryParse(Console.ReadLine(), out int sonuc);
    return sonuc; // Kullanıcının girdiği sayıyı teslim eder
}

// Çağrılışı:
int yas = SayiAl("Kaç yaşındasınız?");
int plaka = SayiAl("Memleketinizin plakası kaç?");
```

3. Parametre Aktarım Mekanizmaları: Değer vs. Referans

Metotların en çok karıştırılan ve mülakatlarda en sık sorulan konusu parametrelerin bellekte nasıl taşındığıdır.

```
1. Değer ile Aktarım (Varsayılan):
   Main (x = 10) ——— Fotokopisini Gönder ———► Metot (x = 20)
```

```
   (Main'deki orijinal x DEĞİŞMEZ, 10 kalır)

2. Referans ile Aktarım (ref / out):
   Main (x = 10) [Adres: 0x4A] ◄── Aynı Kutuyu Gösterir ──► Metot (x = 20) [Adres: 0x4A]
   (Metot kutunun içini değiştirince Main'deki orijinal x de DEĞİŞİR, 20 olur)
```

A. Varsayılan Davranış: Değer ile Aktarım (Pass by Value)

C#'ta temel veri tipleri (`int`, `double`, `bool` vb.) bir metoda gönderildiğinde değişkenin kendisi değil, o anki değerinin bir **kopyası (fotokopisi)** metoda gider. Metot içinde yapılan değişiklik orijinal değişkeni etkilemez.

C#

```
int sayi = 10;
Degistir(sayi);
Console.WriteLine(sayi); // Çıktı yine 10'dur! Değişmedi.

void Degistir(int x)
{
    x = 50; // Yalnızca kopyası değişti
}
```

B. `ref` Anahtar Kelimesi (Pass by Reference)

Değişkenin kopyası yerine **bellekteki doğrudan adresi (orijinal kutusu)** metoda verilir. Metodun içeride yaptığı her değişiklik, çağıran yerdeki asıl değişkeni anında ve kalıcı olarak değiştirir.

- **Kural:** `ref` olarak gönderilecek değişkenin metoda girmeden önce **mutlaka bir ilk değeri (başlangıç değeri)**atanmış olmalıdır.

C#

```
int sayi = 0;
DegistirRef(ref sayi);
Console.WriteLine(sayi); // Çıktı: 20 oldu! Orijinal değişken değişti.

void DegistirRef(ref int x)
{
    x = 20;
}
```

C. `out` Anahtar Kelimesi (Dışarıya Veri Fırlatma)

`ref` ile benzer şekilde adresi aktarır; ancak kullanım amacı **metottan birden fazla değer döndürmektir** (Normalde bir metot `return` ile sadece tek bir değer dönebilir).

- **Kural 1:** Metoda girmeden önce değişkene ilk değer verme zorunluluğu **yoktur**.
- **Kural 2:** Metot kendi içinde bu değişkene **mutlaka bir değer atamak zorundadır**; aksi takdirde derleyici hata verir.

C#

```
// Hem bool (başarılı mı?) hem de out ile toplam ve mesaj dönen yapı:
bool basarili = Toplam(out int sonuc, 10, 20, out string bilgiMesaji);

Console.WriteLine($"Durum: {basarili}, Sonuç: {sonuc}, Not: {bilgiMesaji}");

bool Toplam(out int toplam, int a, int b, out string message)
{
    if (a > 0 && b > 0)
    {
        toplam = a + b; // Atama zorunludur!
        message = "İşlem Başarılı";
        return true;
    }
    else
    {
        toplam = 0;    // Atama zorunludur!
        message = "Sayılar pozitif olmalıdır!";
        return false;
    }
}
```

> **Gündelik Hayattan `out` Örneği:**
>
> Daha önce kullandığımız `int.TryParse(Console.ReadLine(), out int sayi)` yapısı tam olarak budur. Metot `bool` ile dönüşümün başarılı olup olmadığını dönerken, çevrilen gerçek sayıyı `out` parametresiyle teslim eder.

D. `in` Anahtar Kelimesi (Salt Okunur Referans - Read Only Reference)

Veriyi bellek performansı sağlamak için kopyalamadan referansıyla (adresiyle) gönderir; ancak metodun bu değişkeni **değiştirmesini kesin olarak yasaklar**.

- Metot içinde değişken sadece okunabilir, yeni bir değer atanamaz (`readonly`).

C#

```
bool DaireAlanHesapla(out double sonuc, double r, in double pi)
{
    // pi = 3.15; // DERLEME HATASI VERİR! 'in' parametresi değiştirilemez.
    if (r <= 0)
    {
        sonuc = 0;
        return false;
    }
    sonuc = (r * r) * pi;
    return true;
}
```

`ref` vs `out` vs `in` Özet Tablosu

Belirteç	Metoda Girmeden Önce Değer Zorunlu mu?	Metot İçinde Değer Atamak Zorunlu mu?	Metot Değeri Değiştirebilir mi?	Temel Amacı
`ref`	**Evet**	Hayır	Evet	Mevcut değişkeni metot içinde güncellemek
`out`	**Hayır**	**Evet**	Evet	Metottan dışarıya ek sonuç(lar) çıkarmak
`in`	**Evet**	Hayır	**Hayır (Yasak)**	Büyük veriyi kopyalamadan, güvenle okumak

4. İleri Seviye Metot Parametre Özellikleri

A. İsteğe Bağlı / Varsayılan Parametreler (Optional / Default Parameters)

Bir parametreye metot tanımlanırken varsayılan bir değer atanabilir. Çağıran taraf o parametreyi göndermezse varsayılan değer devreye girer.

- **Kural:** Varsayılan değerli parametreler parametre listesinin **en sonunda** yer almak zorundadır.

C#

```
void MesajYaz(string isim, string mesaj = "Merhaba", int tekrar = 1)
{
    for (int i = 0; i < tekrar; i++)
    {
        Console.WriteLine($"{mesaj} {isim}");
    }
}

// Farklı çağrılış şekilleri:
MesajYaz("Berker");                          // "Merhaba Berker" (1 kez)
MesajYaz("Berker", "Günaydın");              // "Günaydın Berker" (1 kez)
MesajYaz("Berker", "İyi akşamlar", 3);       // "İyi akşamlar Berker" (3 kez)
```

B. Değişken Sayıda Parametre Alma: `params` Anahtar Kelimesi

Bir metoda aynı türden kaç adet argüman gönderileceği önceden bilinmiyorsa (2 tane de gelebilir, 20 tane de) `params` kullanılır. Metot bu girdileri otomatik olarak bir diziye (`array`) toplar.

- **Kural:** Bir metotta yalnızca **bir adet** `params` olabilir ve listenin **en son parametresi** olmalıdır.

C#

```
int Topla(params int[] sayilar)
{
    int toplam = 0;
    foreach (var n in sayilar)
    {
        toplam += n;
    }
    return toplam;
}

// İstediğimiz kadar argümanla çağırabiliriz:
Console.WriteLine(Topla(4, 6));               // 10
```

```
Console.WriteLine(Topla(4, 6, 8, 2));          // 20
Console.WriteLine(Topla(1, 2, 3, 4, 5, 6, 7)); // 28
```

5. Metot Tabanlı Modüler Mimari (Hesap Makinesi Örneği)

Ders kodundaki hesap makinesi örneği, **sorumlulukların ayrılması (Single Responsibility)** ilkesini çok temiz yansıtır.Her görev ayrı bir metoda verilmiştir:

```
[Kullanıcı Girişi: "489+653"]
             │
             ▼
      metinOku()  ──> Kullanıcıdan metni güvenle okur
             │
             ▼
     sayilariBol() ──> Hangi işlemin yapılacağını tespit eder (+, -, *, /)
             │
             ├──► sayiyaCevir() ──> String parçalarını int dizisine dönüştürür
             │
             └──► toplam() / cikar() / carp() / bol() ──> Yalnızca matematiksel işlemi ya
par
```

C#

```
// Sorumluluk 1: Konsoldan metin okuma
string MetinOku(string mesaj)
{
    Console.WriteLine(mesaj);
    return Console.ReadLine() ?? string.Empty;
}

// Sorumluluk 2: String dizisini sayı dizisine dönüştürme
int[] SayiyaCevir(string[] parcalar)
{
    if (parcalar.Length != 2) return null;

    int[] sayilar = new int[2];
    for (int i = 0; i < parcalar.Length; i++)
    {
        sayilar[i] = int.Parse(parcalar[i].Trim());
    }
    return sayilar;
}

// Sorumluluk 3: Temel matematik işlemleri
```

```
int Topla(int a, int b) => a + b;
int Cikar(int a, int b) => a - b;
int Carp(int a, int b)  => a * b;
int Bol(int a, int b)   => b != 0 ? a / b : 0;
```

6. Kapsamlı Uygulama Analizi: Sayısal Loto Simülasyonu

Ders kodunun sonundaki Sayısal Loto uygulaması;
diziler, `Random`, `ref`, `out`, `while`, iç içe döngüler ve yardımcı metotları bir araya getiren algoritma tasarımıdır.

Sistemin Algoritmik Akışı

1. **Benzersizlik Kontrolü (`isOne` Metodu):**C#

   Üretilen veya girilen sayının dizide daha önce bulunup bulunmadığını kontrol eder (Mükerrer kaydı engeller).

   ```
   bool IsUnique(int sayi, int[] dizi)
   {
       foreach (var eleman in dizi)
       {
           if (eleman == sayi && eleman > 0)
               return false; // Sayı zaten dizide var!
       }
       return true; // Sayı benzersiz, eklenebilir.
   }
   ```

2. **Dinamik Veri Alma (`TahminAl` Metodu):**

   Tek bir metot `bool isSistem` parametresi ile hem kullanıcının klavyeden sayı girmesini hem de bilgisayarın rastgele 6 benzersiz sayı çekmesini sağlar.

3. **Eşleşme Sayma ve Kazanma Kontrolü (`Control` Metodu):**

   İki diziyi iç içe döngüyle karşılaştırır, kaç tane ortak eleman olduğunu `out int sayi` ile dışarı bildirir, 5 veya daha fazla bilindiyse `true` döner.

C#

```
bool Control(int[] tahmin, int[] sistem, out int bilinenAdet)
{
    int count = 0;
    for (int i = 0; i < sistem.Length; i++)
    {
        for (int j = 0; j < tahmin.Length; j++)
        {
            if (sistem[i] == tahmin[j])
            {
                count++;
                break; // Eşleşen sayı bulundu, diğer tahminlere bakmaya gerek yok
            }
        }
    }
    bilinenAdet = count;
    return count >= 5; // 5 veya 6 bildiyse büyük ödül
}
```

7. Kodlama Standartları ve İyi Alışkanlıklar (Best Practices)

- **Metot Adlandırma:** Metot isimleri eylem bildirmelidir (`VeriGetir`, `Hesapla`, `Kaydet`).
- **Tek Bir İş Yap:** Bir metot hem ekrandan veri alıp, hem veritabanına kaydedip, hem de fatura yazdırmamalıdır. Her metot tek bir göreve odaklanmalıdır (*Single Responsibility*).
- **Gereksiz `ref` Kullanımından Kaçın:** Diziler (`int[]`) zaten C#'ta referans tiplidir (Heap'te yaşar). Dizi elemanlarını değiştirmek için metoda ayrıca `ref int[]` yazmaya gerek yoktur; dizinin kendisini parametre vermek yeterlidir. `ref` sadece dizinin boyutunu metot içinde `new` ile sıfırdan yeniden oluşturmak istediğinizde zorunludur.